



Boas Práticas de Programação Advpl

Manual de Regras e Padronização

Boas Práticas de Programação.....	1
Para que padronizar?.....	6
Legibilidade de Código.....	7
Estrutura de um programa	7
Área de cabeçalho	8
Área de Identificação	8
Área de Ajustes Iniciais	9
Corpo do Programa	9
Área de Encerramento.....	9
Linhas de Comentário.....	10
Tamanho da Linha.....	11
Utilização de Espaços em Branco	11
Não abreviar comandos	12
Utilização de Identação.....	12
Capitulação de Palavras-Chave.....	13
Palavras em maiúsculo	14
Utilização da Notação Húngara	14
Nomeando um Código Fonte.....	15
Regras Básicas de Programação	16
Variáveis	16
Declaração.....	16
Visibilidade	16
Inicialização	16
Funções.....	17
Declaração.....	17
Visibilidade	17
Nomeando	17
Palavras Reservadas	18
Gravando variáveis ambientais	19
Retorno de funções.....	19
Utilizando loops	20
Loops Infinitos.....	20
Trabalhando com Dados.....	21
Trabalhando com Registros	21
Como referenciar um campo.....	21
Cuidados com Posicionamentos de Registros	21
dbSeek()	21

SoftSeek.....	21
Funções de Posicionamento Restritas.....	22
Funções de Procura.....	22
Posicione().....	22
ExistCpo().....	22
Travamentos / Bloqueios / Locks.....	23
RecLock(cAlias, lAppend).....	23
MSUnlock(cAlias).....	23
Funções de Travamento Restritas	24
DBRLock([recno]).....	24
DBRUnlock([recno]).....	24
MSRLock([recno]).....	24
MSRUnlock.....	24
DBUnlock.....	24
DBUnlockAll.....	24
MultLock(Alias, aChaves, nOrd).....	24
SoftLock.....	25
MSUnlockAll.....	25
MSUnlockSoft.....	25
DeadLock	26
Como evitar	26
Leitura Suja	27
Cuidados	27
Controle de Transação (TTS)	28
O que é.....	28
Quando usar	28
Como usar.....	28
BEGIN TRANSACTION...END TRANSACTION.....	28
FKCommit().....	29
Onde não usar.....	29
Trabalhando com arquivos (Tabelas de Dados).....	30
Conceito de Filial e Compartilhamento de Arquivos.....	30
xFilial().....	31
cFilAnt e cEmpAnt.....	32
Arquivos e Índices Temporários.....	33
CriaTrab.....	33
IndRegua.....	33
Criando e Deletando Arquivos temporários.....	34
Usando Filtros	35
Querys – Embedded SQL.....	36
Dicas de Tunning.....	37
Integridade Referencial	38
Chaves Primárias.....	38
Chaves Estrangeiras.....	38
Dicas Importantes	38
Sistema Internacionalizado	39

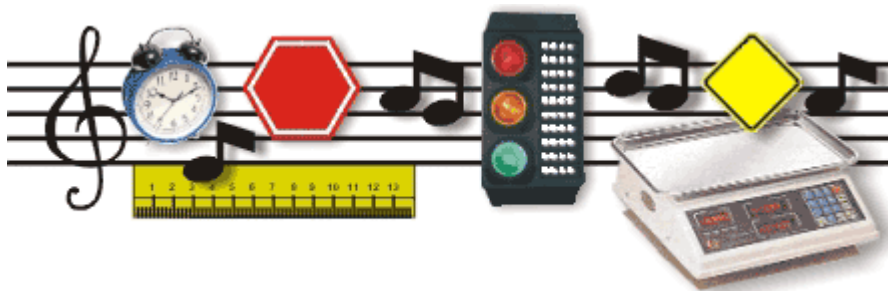
Diferença entre Localizar e Traduzir	39
Localizar	39
Traduzir	39
Como produzir um programa traduzido?	40
Arquivos “header” (CH) anexados ao código-fonte.....	40
Dicionário de Dados	41
Como Produzir um programa Localizado	42
Processamento Automático.....	43
Rotinas Automáticas	43
O que são?	43
Como fazer?	43
Schedule	44
Processos de Integração.....	45
Customizações	46
Pontos de Entrada	46
O que são?	46
Quando criar?	46
Utilização.....	46
Semáforo	47
Desenvolvendo Telas.....	48
Interfaces do Protheus	48
Browses	49
mBrowse()	49
MarkBrow()	50
TWBrowse().....	51
Perguntas	52
Pergunte()	52
Entrada de Dados.....	53
Enchoice()	53
MSGetDados()	55
Componentes Individuais	56
TSay()	56
TGet()	56
TComboBox() e TListBox()	56
TCheckBox()	56
TButton() e SButton()	57
TPanel().....	57
Mensagens.....	58

Aviso()	58
Help()	58
MsgNoYes(), MsgStop(), MsgInfo() e MsgAlert()	59
MSAguarde()	59
MsNewProcess()	59
Outros Modelos	60
dbTree()	60
APWizard()	61
Ícones e Legendas	62
Legendas	62
Ícones	62
Desenvolvendo Relatórios	63
Relatórios Gráficos	63
Pecados da Programação	64
Excesso de Refresh	64
SET CENTURY / DATE FORMAT e Loops	64
Begin...End Sequence / Break	64
Interface durante transação	65
Transações muito longas	65
Alto acopalhamento com Interface	65
dbGoTop	65
Fontes com “economia” de IF	65
“*” em query’s	66
Objetos visuais	66
Objetos visuais em loops	66
Objetos visuais e o End()	66
Objetos visuais em Jobs	67
MSAdvSize() e dimensões de janela	67
Codeblocks em componentes visuais	67
DEFINE DIALOG e ACTIVATE	67
Funções em ON INIT	67
DBTree	67

Para que padronizar?

O ser humano convive com a padronização há milhares de anos e depende dela para a sua sobrevivência, mesmo que não tenha consciência disto.

Imagine como seriam as relações comerciais entre as nações se não existisse o Sistema Métrico para estabelecer uma linguagem comum? Ou então, como seria possível manter a ordem pública sem os sinais de trânsito?



A padronização deve ser vista dentro das organizações da mesma forma, ou seja, como algo que trará benefício para todos: diretores, gerentes, executantes, fornecedores e clientes.

Hoje, com a complexidade dos processos produtivos e gerenciais, mais do que nunca é necessário registrar de forma organizada (em meio físico ou eletrônico) a maneira de se trabalhar e introduzir formalmente o treinamento no trabalho (On the Job Training - OJT).

Podemos, então, definir **PADRÃO** como sendo:

"Compromisso documentado, utilizado em comum e repetidas vezes pelas pessoas relacionadas com um determinado trabalho."

As principais vantagens de se padronizar o desenvolvimento de sistemas numa organização são:

- Disciplina nos métodos de trabalho;
- Facilidade de controles e conseqüente gerenciamento;
- Diminuição dos problemas de manutenção.

Padronizar ⇒ Servir de Modelo

Legibilidade de Código

Entende-se por legibilidade de código, a facilidade de ler e entender o que foi escrito pelo programador. Usando as regras de legibilidade de código, fica fácil para outro programador entender os fontes e facilitará futuras alterações feitas por qualquer programador.

Estrutura de um programa

Um programa em Advpl pode ser dividido em 5 partes básicas:

- A) Área de cabeçalho
- B) Área de identificação
- C) Área de declaração de variáveis e ajustes iniciais
- D) Corpo do programa
- E) Área de encerramento

```

A      #Include "Protoheus.ch"
      #Include "PrgExem.ch"
      /*

      Programa | PrgExem | Microsiga | Data | xx.xx.xx
      Descrição | Programa de exemplo -> Entrada de Valor
      Sintaxe | PrgExem()
      Parametros
      Retorno | nValor = Valor Informado
      Uso | Generico

      ATUALIZACOES SOFRIDAS DESDE A CONSTRUÇÃO INICIAL.
      Programador | Data | BOPS | Motivo da Alteracao
      | | | |
      | | | |

      */
      Function PrgExem()

C      Local nValor := 0
      Local oDlg

      DEFINE MSDIALOG oDlg FROM 31,15 TO 510,574 TITLE OemToAnsi(STR0001) PIXEL OF oMainWnd //"Entrada de Valor"

D      @ 010, 010 SAY OemToAnsi(STR0002) SIZE 18,07 OF oDlg PIXEL //"Informe Valor"
      @ 010, 030 MSGET nValor SIZE 57,10 OF oDlg PIXEL Valid{nValor>0}

      DEFINE SBUTTON FROM 06, 246 TYPE 1 ENABLE OF oDlg ACTION { nOpcl := 1,oDlg:End() }
      DEFINE SBUTTON FROM 20, 246 TYPE 2 ENABLE OF oDlg ACTION { nOpcl := 0,oDlg:End() }

      ACTIVATE MSDIALOG oDlg

E      Return {nValor}
    
```

Se um arquivo de código criado se referencia a comandos para interpretação e tratamento de arquivos XML, este deve se incluir o arquivo de cabeçalho próprio para tais comandos (XMLXFUN.CH no exemplo). Porém não deve-se incluir arquivos de cabeçalho apenas por segurança. Se não se está referenciando nenhuma das constantes ou utilizando nenhum dos comandos contidos em um destes arquivos, a inclusão apenas tornará a compilação mais demorada.

Nesta área também devem declaradas as variáveis estáticas, as constantes e os arquivos ".CH".

Área de Identificação

Esta é uma área dedicada a documentação do programa / função. Contém comentários explicando a sua finalidade, data de criação, parâmetros, retornos e alterações efetuados.

Existem dois tipos de cabeçalho, conforme mostrado a seguir:

■ Cabeçalho de fonte:

/*

Programa		Microsig	Data	xx.xx.xx
Descrição				
Sintaxe				
Parametros				
Retorno				
Uso				
ATUALIZACOES SOFRIDAS DESDE A CONSTRUÇÃO INICIAL.				
Programador	Data	BOPS	Motivo da Alteracao	

*/

■ Cabeçalho de Função:

/*

Funcao		Microsiga	Data	xx.xx.xx
Descrição				
Sintaxe				
Parametros				
Retorno				
Uso				

*/

Área de Ajustes Iniciais

Nesta área devem ser feitos os ajustes iniciais, importantes para o correto funcionamento do programa. Entre esses ajustes iniciais se encontram declarações de variáveis, inicializações, abertura de arquivos etc.

Corpo do Programa

É nesta área que se encontram as linhas de código do programa. É onde se realiza a tarefa necessária através da organização lógica destas linhas de comando. Espera-se que as linhas de comando estejam organizadas de tal modo que no final desta área o resultado esperado seja obtido, seja ele armazenado em um arquivo ou em variáveis de memória, pronto para ser exibido ao usuário através de um relatório ou na tela.

Área de Encerramento

É nesta área onde as finalizações são efetuadas. É onde os arquivos abertos são fechados, e o resultado da execução do programa é utilizado. Pode-se exibir o resultado armazenado em uma variável ou em um arquivo ou simplesmente finalizar, caso a tarefa já tenha sido toda completada no corpo do programa. É nesta área que se encontra o encerramento do programa. Todo programa em AdvPI deve sempre terminar com a palavra chave "Return".

Linhas de Comentário

A formatação permitida para comentários é a seguinte:

```
/*
```

Funcao	Fatorial()	Microsiga	Data	xx.xx.xx
Descrição	Programa para calculo do fatorial de um determinado valor			
Sintaxe	Fatorial(nValor)			
Parametros	nValor = Numero a ter o fatorial calculado			
Retorno	nFatorial = Valor do fatorial			
Uso	Generico			

```
*/
```

```
Function Fatorial(nValor)
```

```
Local nContador := 0 // Contador do laço for
```

```
Local nResultado := 1 // Resultado do fatorial
```

```
//
```

```
//| Calculo do Fatorial:
```

```
//| 2! -> 2 * 1 = 2
```

```
//| 4! -> 4 * 3 * 2 * 1 = 24
```

```
//
```

```
For nContador := nValor to 1 Step -1
```

```
// nResultado deve ser inicializado obrigatoriamente com 1!
```

```
nResultado *= nContador
```

```
Next nContador
```

```
Return (nResultado)
```

Dicas sobre comentários:

- Comente apenas o necessário! Comentários demais “poluem” o fonte, e não trazem ganho significativo à legibilidade do código.
- Passagens complicadas no fonte são fortes candidatos a terem um comentário.
- Cálculos complicados devem ser explicados.
- Pontos de entrada devem ter comentários sobre o seu uso.

Tamanho da Linha

O tamanho máximo ideal de uma linha para visualização na ferramenta IDE é de 130 caracteres. Se a linha digitada ultrapassar esse limite utilize o ponto-e-virgula (;) para dividi-la.

Pode-se também dividir linhas menores que 130 caracteres em mais linhas para tornar o código mais legível. Veja os exemplos abaixo:

```
If !Empty(cNome) .And. !Empty(cEnd) .And. !Empty(cTel) .And. !Empty(cFax).And. nValor !=0
    GravaDados(cNome,cEnd,cTel,cFax,cEmail)
Endif
```

O código acima pode ser reescrito:

```
If !Empty(cNome) .And. !Empty(cEnd) .And. !Empty(cTel) .And.;
    !Empty(cFax) .And. nValor != 0

    GravaDados(cNome,cEnd,cTel,cFax,cEmail)

Endif
```

Utilização de Espaços em Branco

Espaços em branco extras tornam o código mais fácil para a leitura. Não são necessárias imensas áreas em branco, mas agrupar pedaços de código através da utilização de espaços em branco funciona muito bem. Costuma-se também separar parâmetros com espaços em branco. Veja os exemplos abaixo:

```
If !Empty(cNome) .And. !Empty(cEnd) .And. !Empty(cTel) .And.;
    !Empty(cFax) .And. nValor != 0
    GravaDados(cNome,cEnd,cTel,cFax,cEmail)
Endif
```

O código fica mais legível assim:

```
If !Empty(cNome) .And. !Empty(cEnd) .And. !Empty(cTel) .And.;
    !Empty(cFax) .And. nValor != 0

    GravaDados(cNome,cEnd,cTel,cFax,cEmail)

Endif
```

Não abreviar comandos

Embora o AdvPl suporte a abreviação de comandos para quatro letras (“Replace” pode ser escrito como “Repl”) é expressamente proibida a utilização dessa funcionalidade. Isto apenas torna o código mais difícil de ser lido e não torna a compilação mais rápida ou simples.

Utilização de Identação

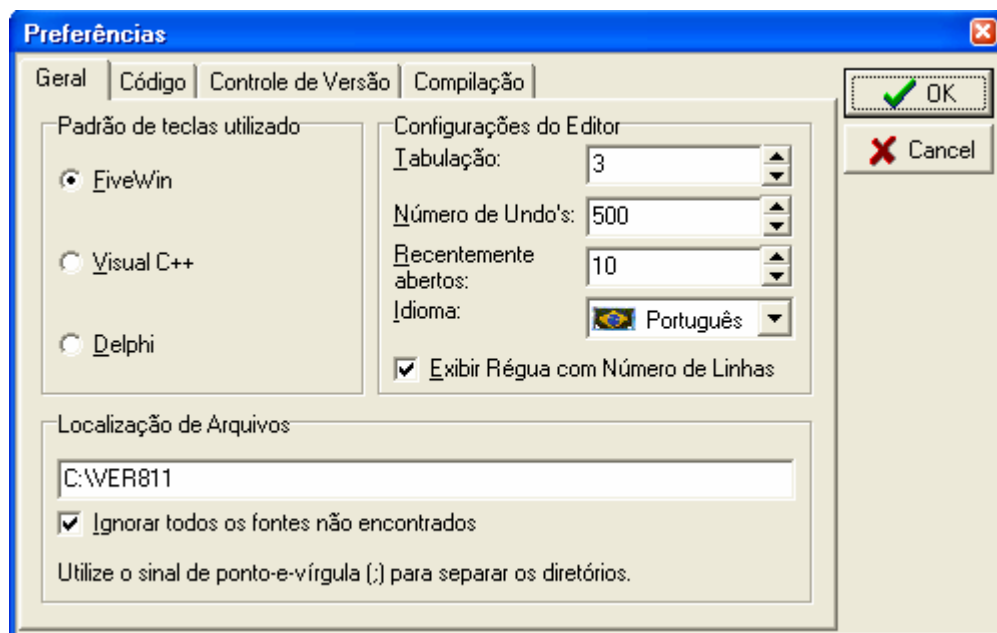
É obrigatória a utilização da indentação, pois torna o código muito mais legível. Veja os exemplos abaixo:

```
While !SB1->(Eof())
If mv_par01 = SB1->B1_COD
dbSkip()
Loop
Endif
Do Case
Case SB1->B1_LOCAL == "01" .Or. SB1->B1_LOCAL == "02"
TrataLocal(SB1->B1_COD,SB1->B1_LOCAL)
Case SB1->B1_LOCAL == "03"
TrataDefeito(SB1->B1_COD)
Otherwise
TrataCompra(SB1->B1_COD,SB1->B1_LOCAL)
EndCase
dbSkip()
EndDo
```

A utilização da indentação seguindo as estruturas de controle de fluxo (while, if, caso etc) torna a compreensão do código muito mais fácil:

```
While !SB1->(Eof())
    If mv_par01 = SB1->B1_COD
        dbSkip()
    Loop
Endif
Do Case
Case SB1->B1_LOCAL == "01" .Or. SB1->B1_LOCAL == "02"
    TrataLocal(SB1->B1_COD,SB1->B1_LOCAL)
Case SB1->B1_LOCAL == "03"
    TrataDefeito(SB1->B1_COD)
Otherwise
    TrataCompra(SB1->B1_COD,SB1->B1_LOCAL)
EndCase
    dbSkip()
EndDo
```

Para indentar o fonte utilize a tecla <TAB> e na ferramenta IDE, configure em “Preferências” o espaçamento da indentação:



Capitulação de Palavras-Chave

Uma convenção amplamente utilizada é a de capitular as palavras chaves, funções, variáveis e campos utilizando uma combinação de caracteres em maiúsculo e minúsculo, visando facilitar a leitura do código fonte. O código a seguir:

```
local ncnt
while ( ncnt++ < 10 )
    ntotal += ncnt * 2
enddo
```

Ficaria melhor com as palavras chaves e variáveis capituladas:

```
Local nCnt
While ( nCnt++ < 10 )
    nTotal += nCnt * 2
EndDo
```

Obs:

Para funções de manipulação de dados que comecem por “db”, a capitulação só será efetuada após o “db”.

```
dbSeek()
dbSelectArea()
```

Palavras em maiúsculo

A regra é utilizar caracteres em maiúsculo para:

■ Constantes:

```
#define NUMLINES 60  
#define Numpages 1000
```

■ Variáveis de memória:

```
M-> CT2_CRCONV  
M->CT2_MCONVER := CriaVar("CT2_CONVER")
```

■ Campos:

```
SC6->C6_NUMPED
```

■ Querys:

```
SELECT * FROM...
```

Utilização da Notação Húngara

A notação húngara consiste em colocar-se prefixos nos nomes de variáveis, de modo a facilmente se identificar seu tipo. Isto facilita na criação de códigos-fonte extensos, pois usando a Notação Húngara, você não precisa ficar o tempo todo voltando à definição de uma variável para se lembrar qual é o tipo de dados que deve ser colocado nela. Variáveis devem ter um prefixo de Notação Húngara em minúsculas, seguido de um nome que identifique a função da variável, sendo que a inicial de cada palavra deve ser maiúscula.

É obrigatória a utilização desta notação para nomear variáveis.

Notação	Tipo de dado	Exemplo
a	Array	aValores
c	Caracter	cNomeFornecedor
d	Data	dDataInicial
l	Lógico	lContinua
n	Numérico	nValorConta
o	Objeto	oMainWindow
x	Indefinido	xBuffer

Nomeando um Código Fonte

O nome do código-fonte deve ser montado da seguinte forma:

Famílias de Programas + Tipo de Operação + Identificador Numérico.prw

O Tipo de operação é indicado na tabela abaixo:

Tipo de Operação	Utilizar:
Entrada de dados / Processamentos	A
Consulta de dados	C
Relatórios	R
Funções genéricas do módulo	X

Exemplo:

Código-fonte de inclusão de dados do módulo Gestão de Pessoas -> **GPEA010.PRW**

GPE	família de programas do módulo Gestão de Pessoas
A	entrada de dados
010	numeração de fonte disponível
.PRW	extensão indicadora de que o arquivo é um código-fonte

Importante!

- Novos fontes devem ter a extensão “.prw”.
- Fontes migrados da versão DOS serão mantidos como “.prx” até que sejam desativados ou reescritos.
- Nova inclusão de família deve ser solicitada para área Engenharia de Software.

Veja Anexo 1 com a Família de programas disponíveis.

Regras Básicas de Programação

Variáveis

Ao se utilizarem variáveis deve-se estar atento para sua:

- Declaração
- Visibilidade
- Inicialização

Declaração

É **obrigatória** a declaração da variável no início da rotina. Deve-se utilizar a notação húngara para nomear as variáveis.

```
Function a910VerCod()  
Local cCod910 := "001"
```

Visibilidade

O seu uso está determinado conforme indicado a seguir:

Proibidas: Public

Restritas: Private (os casos deverão ser avaliados)

Liberadas: Local e Static

Inicialização

Todas as variáveis deverão ser inicializadas no momento de sua declaração. Inicialize com um valor discreto, ou utilize a função CriaVar()*.

***CriaVar():** Esta função cria uma variável, retornando o valor do campo, de acordo com o dicionário de dados. Avalia o inicializador padrão e retorna o conteúdo de acordo com o tipo de dado definido no dicionário.

Funções

Ao se criarem novas funções deve-se estar atento à:

- Declaração
- Visibilidade
- Nomeando
- Passagem de parâmetros
- Recebimento de parâmetros
- Entrada da função
- Saída da função

Declaração

As funções não necessitam nenhuma declaração se forem executadas a partir do menu da aplicação. Caso sejam executadas a partir do Remote, devem ser declaradas como Main ou User Function.

Visibilidade

Funções que são utilizadas somente dentro de um mesmo código-fonte devem ser obrigatoriamente declaradas como STATIC. Esse tipo de função é visível somente no código-fonte em que foi chamada.

Demais tipos de funções são visíveis em todo o sistema e devem ser usadas com critério, pois carregam a pilha de memória.

Nomeando

As funções pertencentes a um código-fonte podem ser nomeadas de acordo com a sua aplicabilidade. Utiliza-se como regra geral a seguinte forma:

- 1ª. Letra do nome do código fonte
- Tipo de Operação indicado no código fonte
- Identificador do código-fonte
- abreviação descritiva do que a função faz

Exemplo:

Função de inclusão de dados presente FINA050 -> FA050Inclu

F	1ª. Letra do nome do código fonte
A	Tipo de operação (manipulação de dados)
050	Identificador do código fonte
Inclu	Abreviação do que a função faz -> inclusão de dados

Para funções genéricas utilizadas em um determinado módulo, nomeia-se a função diretamente de acordo com a sua aplicabilidade.

Exemplo:

SaldoTit() -> calculo saldo de títulos

CalcAbat() -> calculo do abatimento do título

Palavras Reservadas

As palavras reservadas do ADVPL são:

AADD	DTOS	INKEY	REPLICATE	VAL
ABS	ELSE	INT	RLOCK	VALTYPE
ASC	ELSEIF	LASTREC	ROUND	WHILE
AT	EMPTY	LEN	ROW	WORD
BOF	ENDCASE	LOCK	RTRIM	YEAR
BREAK	ENDDO	LOG	SECONDS	CDOW
ENDIF	LOWER	SELECT	CHR	EOF
LTRIM	SETPOS	CMONTH	EXP	MAX
SPACE	COL	FCOUNT	MIN	SQRT
CTOD	FIELDNAME	MONTH	STR	DATE
FILE	PCOL	SUBSTR	DAY	FLOCK
PCOUNT	TIME	DELETED	FOUND	PROCEDURE
TRANSFORM	DEVPOS	FUNCTION	PROW	TRIM
DOW	IF	RECCOUNT	TYPE	DTOC
IIF	RECNO	UPPER	TRY	AS
CATCH	THROW			

Notas:

- Palavras reservadas não podem ser utilizadas para variáveis, procedimentos ou funções.
- Funções reservadas são pertencentes ao compilador e não podem ser redefinidas por uma aplicação.
- Todos os identificadores que comecem com um ou mais caracteres de sublinhado () são utilizados como identificadores internos e são também reservados.

Gravando variáveis ambientais

Para garantir a integridade do sistema é obrigatória a gravação e restauração do ambiente a partir das funções:

- GetArea() -> utilizar no início da função
- RestArea() -> utilizar antes da saída da função

Exemplo:

```
Function Calculox(nValor)
Local IRet := .T.
Local aSaveArea := GetArea()
...
...
...
RestArea(aSaveArea)
Return IRet
```

Retorno de funções

As funções e códigos-fonte devem preferencialmente ter apenas 1 ponto de abandono (retorno). Se existirem vários pontos de “abandono”, utilize variáveis lógicas para compor a estrutura como mostrado no exemplo abaixo:

```
Function ValidCont(cConteudo)

Local aSaveArea := GetArea()
Local IRet := .T.

If Empty(cConteudo)
    Help(" ",1,"NOCONTEUDO")    //"Sem Conteúdo"
    IRet := .F.
ElseIf cTipoTit = "D"
    Help(" ",1,"CONTINCORR")    //"Conteúdo incorreto"
    IRet := .F.
Endif

RestArea(aSaveArea)

Return IRet
```

Utilizando loops

Ao utilizar o comando While não esquecer de incluir a condição referente à filial (quando esta leitura for de registros de uma filial) e de final de arquivo (Eof()).

Exemplo :

```
dbSelectArea("SB1")
dbSeek(xFilial("SB1")+cVar)

Do While ! Eof() .And. SB1->B1_FILIAL == xFilial("SB1")
    // Processamento
    dbSkip()
Enddo
```

Importante!

- A falta do Eof() pode acarretar em um Loop Infinito (vide a seguir).
- A falta da leitura da filial pode acarretar em leitura incorreta de dados (filial errada).

Loops Infinitos

Muito cuidado ao utilizar laços em funções para que o programa não trave por falta de uma saída desse laço. Vide exemplos abaixo:

```
dbSeek(xFilial("SE1")+DTOS(dDtIni))
Do While SE1->(!Eof())
    ...
    ...
    ←----- Falta um dbSkip()
Enddo
```

```
aCampos := { }
Do while .T.
    Aadd(aCampos, "Teste")    ←----- quando vai terminar??
Enddo
```

Trabalhando com Dados

Trabalhando com Registros

Como referenciar um campo

Todos os campos deverão ser referenciados com o seu Alias e em letras maiúsculas:

SB1->B1_FILIAL

SB1->B1_CODPROD

Cuidados com Posicionamentos de Registros

dbSeek()

Ao executar um dbSeek() **SEMPRE** verifique se localizou o registro, exemplo:

Exemplo:

```
If !SB1->(dbSeek(xFilial("SB1")+cVar))  
    // Não achei o registro  
Endif
```

Mesmo que seja óbvio a existência do registro, faça o teste para evitar qualquer interrupção indesejada do programa.

SoftSeek

A função dbSeek() possui a opção de "SoftSeek", isto é, determina se será usada uma busca relativa durante uma procura em um banco de dados. Se nenhuma correspondência for encontrada, o ponteiro de registro ficará no próximo registro do índice que possua um valor mais alto que a expressão utilizada nesta função. Esta opção deverá ser utilizada com a máxima atenção, pois caso esteja ligado, poderá localizar um registro errado.

Funções de Posicionamento Restritas

É **RESTRITA** a utilização das seguintes funções:

- dbGoTop()
- dbSeek(xFilial())

Importante!

- A utilização dessas duas funções só é justificável quando se está utilizando algum filtro de leitura ou um arquivo temporário.
- O dbSeek com a passagem somente de xFilial() deve ser evitado. Se a chave de procura, incluir outros que não somente a Filial, o uso de dbSeek é liberado.

Funções de Procura

É recomendado o uso das funções de procura a seguir:

Posicione()

Podemos também buscar uma informação em determinado campo usando a função Posicione.

Exemplo:

```
cDesc:= Posicione("SB1", 1, xFilial("SB1") + cCodigo, "B1_DESC")
```

Desta forma, será efetuada uma busca no SB1, na ordem 1, chave da busca xFilial("SB1") + cCodigo e será retornado o conteúdo do campo "B1_DESC". Note que esta função, não restaura a posição original do arquivo alvo (no caso SB1).

É necessário passar a filial do arquivo na chave passada como parâmetro, caso ela exista na chave do índice.

ExistCpo()

Retorna se determinada chave existe ou não no arquivo.

Exemplo :

```
ExistCpo("SE1", M->EF_PREFIXO+M->EF_TITULO+M->EF_PARCELA,1)
```

Desta forma, será efetuada uma busca no SE1, na ordem 1, chave:

M->EF_PREFIXO+M->EF_TITULO+M->EF_PARCELA. E será retornado se a chave foi encontrada ou não (.T. ou .F.).

Não é necessário passar a filial. Ela será inserida automaticamente na chave de pesquisa pela própria função.

Travamentos / Bloqueios / Locks

Quando estamos trabalhando em um ambiente multi-usuário, se faz necessário um controle de bloqueio desta informação para quando a mesma for atualizada ou excluída. Esse bloqueio serializa as operações, possibilitando que vários usuários façam alterações no mesmo registro, porém não ao mesmo tempo.

RecLock(cAlias, lAppend)

Tem a função de criar um registro em branco para inserção ou bloquear o registro atual para edição, neste caso a função executa um refresh do dado, buscando a informação mais atual no banco. Durante o período que o registro estiver bloqueado os demais usuários podem acessá-lo apenas para consulta (vide adiante – DeadLock - leitura suja).

Caso não seja possível o bloqueio do registro a função irá interagir com o usuário, questionando se deve permanecer tentando o bloqueio ou desistir da operação.

Exemplo:

Verifica se o registro existe na tabela SA1:

```
If !dbSeek(xFilial("SA1")+"000001")
    // Se não existir, insere um registro em branco e o bloqueia
    Reclock("SA1", .T. )
Else
    // Bloqueia o registro encontrado
    Reclock("SA1", .F. )
Endif
```

MSUnlock(cAlias)

Libera o registro criado ou bloqueado pela RecLock.

Exemplo:

Verifica se o registro existe na tabela SA1:

```
If !dbSeek(xFilial("SA1")+"000001")
    //Se não existir, insere um registro em branco e o bloqueia
    Reclock("SA1", .T. )
Else
    //Bloqueia o registro encontrado
    Reclock("SA1", .F. )
EndIf
SA1->A1_SALDO := nNovoSaldo
MSUnlock("SA1")
```

A recomendação é que sejam utilizadas somente as funções RecLock e MsUnlock. Demais funções de travamento disponíveis só poderão ser utilizadas com aprovação da Engenharia de Software.

Funções de Travamento Restritas

DBRLock([recno])

Realiza o bloqueio do registro indicado pelo parâmetro. Caso este seja omitido irá bloquear o registro corrente, desbloqueando demais bloqueados.

DBRUnlock([recno])

Desbloqueia o registro indicado pelo parâmetro. Caso este seja omitido irá desbloquear o registro corrente.

MSRLock([recno])

Esta função é uma variação da DBRLock. Seu comportamento difere somente no caso do argumento ser omitido, pois ela irá bloquear o registro corrente sem desbloquear os demais bloqueados.

MSRUnlock

Esta função é um encapsulamento da DBRUnlock()

DBUnlock

Desbloqueia **todos** os registros bloqueados para a tabela corrente.

DBUnlockAll

Desbloqueia **todos** os registros bloqueados de **todas** as tabelas abertas pelo sistema.

MultLock(Alias, aChaves, nOrd)

Esta função reserva através de semáforo apropriado as chaves definidas no array aChaves evitando assim a ocorrência de deadlock.

Deve ser utilizada toda vez que houver a necessidade de alterar mais de uma linha de uma mesma tabela dentro da mesma transação.

Um exemplo prático é a alteração de saldos de estoque na inclusão de várias movimentações (na mesma transação).

No caso do TopConnect este “semáforo” é feito através de locks virtuais (em memória), não sendo feito lock no banco. Para outras bases o “semáforo” é feito através de lock de registro.

```
nPosPrd:=aScan(aHeader,{|x| AllTrim(x[2]) == "C6_PRODUTO"})
nPosLoc:=aScan(aHeader,{|x| AllTrim(x[2]) == "C6_LOCAL"})
For ni := 1 to Len(aCols)
    AADD(aTrava, C6_PRODUTO+C6_LOCAL )
Next
lTrava := MultLock("SB2",aTrava,1) .And.;
    MultLock("SA2",{M->C5_CLIENTE+M->C5_LOJACLI},1) .And.;
    MultLock("SA2",{M->C5_CLIENTE+M->C5_LOJAENT},1)
```


SoftLock

A função SoftLock tem por objetivo efetuar um bloqueio no registro quando a operação de alteração ou exclusão for executada pela Mbrowse ou pela MarkBrowse.

Essa função não deve ser utilizada nos programas, visto que se trata de uma função interna, utilizada nas bibliotecas do ADVPL.

MSUnlockAll

Esta função desbloqueia todos os registros bloqueados pelas funções Multlock, SoftLock e RecLock.

MSUnlockSoft

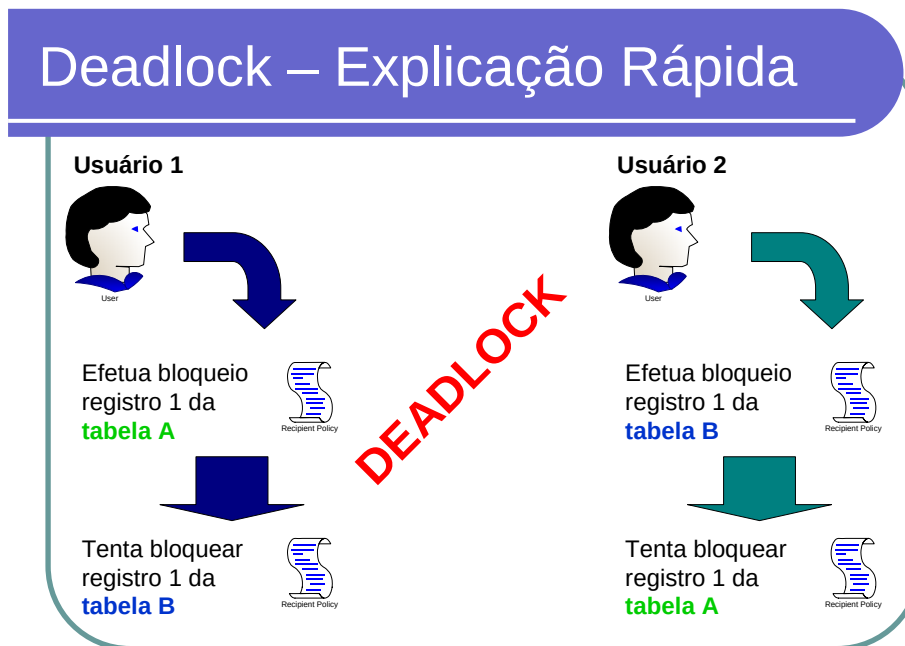
Tem por função desbloquear os registros bloqueados pela SoftLock.

DeadLock

Um sistema está em estado de deadlock quando existe uma operação (A) fazendo um bloqueio em um registro (R1) e tentando bloquear outro registro (R2). Neste mesmo momento existe outra operação (B) bloqueando o registro (R2) e tentando bloquear o registro (R1).

Nesta situação não existe como o banco resolver as solicitações, então ele elege, aleatoriamente, uma das conexões e a encerra.

Exemplo



Como evitar

Dentro do sistema Protheus existem duas formas de evitarmos o deadlock. A primeira delas é utilizando a função Multlock (descrita anteriormente) para bloquear todos os registros que serão manipulados, antes de iniciar a gravação.

A segunda forma, e mais indicada, seria a inserção dos dados sempre na mesma ordem, por exemplo, ao inserir um pedido de venda, o programa deve ordenar os itens pelo código do produto, evitando assim a possibilidade de um deadlock nas tabelas relacionadas ao produto.

Lembre-se sempre de seguir a ordem de dependência da informação, incluindo primeiro as informações “pai” para em seguida incluir as informações “filho”.

Ex. Pedido de venda – incluir cabeçalho depois itens.

Leitura Suja

Permite que outros usuários façam a leitura dos dados que estão bloqueados por outra(s) sessão(ões) e que ainda não foram confirmados. Trata-se do nível mais baixo de consistência de leitura.

Porque utilizar leitura suja

Para permitir que outros usuários tenham acesso aos dados que estão sendo alterados, caso contrário o sistema funcionaria de forma serializada, inviabilizando vários processos.

Impacto da utilização

O grande impacto que pode haver na utilização da leitura suja é a aparição de “fantasmas”, ou seja, a transação T2 lê um dado que estava bloqueado por T1. T1, por sua vez, altera o conteúdo da linha ou até mesmo apaga a linha. Quando T2 for processar alguma informação referente a esta linha ela não existe mais ou seu conteúdo não é mais aquele que foi lido anteriormente.

Cuidados

A situação descrita neste exemplo está incorreta:

```
SA1->MSSeek( "01" + "12345601" )
nVal := SA1->A1_SALDO
Reclock( "SA1", .F. )
nVal += nValAcumulado
SA1->A1_SALDO := nVal
MSUnlock()
```

Existe um erro de lógica, pois a variável nVal recebeu o conteúdo de A1_SALDO sem que o registro estivesse bloqueado. Neste caso o conteúdo de A1_SALDO pode ter sofrido alteração e o conteúdo de nVal está diferente.

A forma correta de escrita do exemplo anterior seria:

```
If SA1->MSSeek( "01" + "12345601" )
    Reclock( "SA1", .F. )
    nVal := SA1->A1_SALDO
    nVal += nValAcumulado
    SA1->A1_SALDO := nVal
    MSUnlock()
Endif
```

Controle de Transação (TTS)

O que é

A seqüência de um ou mais comandos de manipulação de dados constitui uma transação. Por conceito, uma transação necessita de confirmação (COMMIT) ou cancelamento (ROLLBACK) para confirmar ou não as alterações.

O exemplo mais fácil para entendermos o que é uma transação é partindo para uma transferência bancária entre contas. A transação só é confirmada quando a operação for realizada nas duas contas (crédito e débito). Caso uma das duas partes falhe a transação é cancelada.

Quando usar

Quando temos uma operação que necessite que várias inclusões, alterações ou exclusões só sejam efetuadas quando todas as operações tenham sido realizadas com sucesso, garantindo com isso que não sejam atualizadas parcialmente as tabelas envolvidas.

Como usar

BEGIN TRANSACTION...END TRANSACTION

Para definir uma transação, deve-se utilizar os comandos BEGIN TRANSACTION e END TRANSACTION para definir início e fim de uma transação respectivamente. Todas informações à serem gravadas no Banco devem estar dentro de uma única transação sejam elas provenientes de uma ou várias tabelas.

BEGIN TRANSACTION

 ExpN1 :=FuncGrava()

END TRANSACTION

Caso exista uma transação dentro de outra, a segunda será automaticamente ignorada, fechando-se a transação principal quando da chamada do comando END TRANSACTION.

BEGIN TRANSACTION

 FuncGrava1()

BEGIN TRANSACTION

 ExpN1 :=FuncGrava()

END TRANSACTION

END TRANSACTION

←----- é ignorada

←----- é ignorada

Importante!

Dentro de uma transação é proibida a utilização de Exit que interrompa o fluxo de gravações, deixe pendente a transação aberta.

FKCommit()

Executa os comandos de gravação dos dados pendentes no banco (flush) para a tabela corrente, se a integridade referencial estiver ativa. Deve ser utilizada após a inserção de dados em uma tabela pai, em que na mesma transação serão inseridos os filhos.

Exemplo:

```
BEGIN TRANSACTION
  RecLock("SC5",.T.)
  SC5->C5_NUM := cNumPed
  FKCommit()
  For nContador := 1 To Len (aItens)
    RecLock("SC6",.T.)
    SC6->C6_NUM := cNumPed
    SC6->C6_ITEM := cItemPed
    SC6->C6_COD := cProduto
    FKCommit()
  Next nContador
  SC5->(MsUnlock())
END TRANSACTION
```

A execução do comando de gravação no banco de dados pode não ocorrer no mesmo momento em que é executado um MsUnlock(). Por questões de desempenho, o Protheus faz um cachê desses comandos e de tempos em tempos os aplica no banco. Esta execução pode ser antecipada pelas seguintes ações:

- Desposicionamento do ponteiro de registro da tabela que teve a inserção.
- Execução de: FKCommit(), Recno() ou TCQuery().

Onde não usar

É proibida a utilização de laços (WHILE, FOR) dentro de uma transação, pois a área de LOG do banco é limitada, e o volume de informações pode ultrapassar o limite do banco de dados. Por exemplo, devemos controlar a transação de uma nota e não de um conjunto ilimitado de notas para não ultrapassarmos o limite do Log do banco de dados.

O controle de transação jamais deverá ser utilizado durante processos que envolvam interfaces de entrada de dados. O controle deve se resumir apenas ao processo de gravação. Entre um início de transação (Begin Transaction) e um final (End Transaction) todos os registros a serem gravados ficam "bloqueados" até o final da transação. Caso exista uma tela entre o BEGIN e o END, a aplicação fica dependente do usuário para efetuar a liberação da transação, fato que poderia causar muitos transtornos aos usuários.

Rotinas de reproprocessamentos ou recálculos, onde as informações podem ser regeradas durante sua execução não devem ter controle de transação, pois a quantidade de registros processados pode ultrapassar o limite do log de transação.

Trabalhando com arquivos (Tabelas de Dados)

Conceito de Filial e Compartilhamento de Arquivos

O compartilhamento de arquivos é baseado no conceito de filiais. Cada empresa cadastrada pode ter n filiais onde os dados inseridos por cada filial - como cadastro de produtos, clientes, etc. - podem ou não ser compartilhados com as demais filiais. Este recurso somente é possível devido à existência do campo "XX_FILIAL" ("XX" representa o nome do arquivo) em todos os arquivos do sistema.

O modo de operação de cada arquivo - compartilhado ou exclusivo - está definido no arquivo SX2 do dicionário de dados.

Caso o modo de acesso seja compartilhado, quando um novo registro for inserido, o campo XX_FILIAL receberá o valor " " (dois caracteres brancos) e o seu conteúdo será visível por qualquer usuário de qualquer filial.

Caso o modo de acesso seja exclusivo, quando um novo registro for inserido, o campo receberá o código - alfanumérico - da filial (no exemplo abaixo, "01" ou "02") e será visível apenas para os usuários da filial que inseriu o mesmo.

Exemplo:

O arquivo XX está com o modo de acesso definido como exclusivo no SX2, portanto seus registros serão visíveis somente para a filial que os inseriu. Vide exemplo abaixo:

XX_FILIAL	XX_CODIGO	XX_DESCRI
01	30	CHAVE DE FENDA
01	22	CHAVE PHILLIPS
02	21	FURADEIRA BOSCH
02	12	FURADEIRA BLACK & DECKER
01	31	DREMEL
01	90	SERRA TICO-TICO
02	48	PRUMO
02	13	SERROTE

 Usuário Filial 01	 visualiza	22 CHAVE PHILLIPS
		30 CHAVE DE FENDA
		31 DREMEL
		90 SERRA TICO-TICO

 Usuário Filial 02	 visualiza	12 FURADEIRA BLACK & DECKER
		13 SERROTE
		21 FURADEIRA BOSCH
		48 PRUMO

Os usuários da Filial 01 terão acesso somente aos dados cujo registro possua o conteúdo "01" no campo XX_FILIAL. Já os usuários da Filial 02, acessarão os dados dos registros com o valor "02" no campo XX_FILIAL.

xFilial()

Para que o registro realmente fique disponível ou não para suas respectivas Filiais, TODAS as rotinas que manipulam registros diretamente na base de dados deverão verificar a Filial através da Função xFilial().

A função xFilial() verifica se o arquivo é exclusivo ou compartilhado e irá retornar “ ” se o arquivo for Compartilhado ou o código da Filial se o arquivo for exclusivo .

Exemplo :

■ Para executar um dbSeek no arquivo de clientes :
dbSelectArea(“SA1”)
dbSeek(xFilial(“SA1”)+cCodCli+cLoja)

■ Para efetuar um processamento em um arquivo :
While !Eof() .And. CT1_FILIAL=xFilial(“01”)

Importante!!

- O campo XX_FILIAL faz parte da chave de todos os índices do sistema
- Jamais use um campo filial de uma tabela para executar um dbSeek() em outra tabela. Pois uma tabela poderá ser compartilhada (campo filial em branco), enquanto que a outra poderá ser compartilhada (campo filial preenchido).

cFilAnt e cEmpAnt

Estas variáveis de ambiente contêm respectivamente a Filial e a Empresa que o usuário está operando no momento.

Se a rotina for manipular a filial e/ou empresa correntes, deve-se inicialmente gravar a filial e a empresa corrente, para que ao término da rotina, tudo seja restaurado à sua posição inicial.

Exemplo:

```
cSvEmpAnt := cEmpAnt
```

```
cSvFilAnt := cFilAnt
```

```
..... (processamento de código que altera o valor da filial e/ou empresa corrente)
```

```
cEmpAnt := cSvEmpAnt
```

```
cFilAnt := cSvFilAnt
```


Arquivos e Índices Temporários

Arquivos e índices temporários **devem ser utilizados com cuidado**, pois podem gerar um tempo de resposta longo enquanto estão sendo construídos.

No caso de índices temporários, o processamento da rotina é bem mais rápido (após a sua geração), mas de qualquer forma, sempre dê preferência à utilização dos índices-padrão do sistema ou de queries.

Utilize as funções:

CriaTrab

CriaTrab(cAlias,.F.) -> Cria somente um arquivo de índice temporário

CriaTrab(cAlias,.T.) -> Cria um arquivo de dados e um arquivo de índice temporário (a criação do índice temporário não é obrigatória).

IndRegua

Cria efetivamente o índice, a partir do arquivo já criado com a CriaTrab.

Exemplo:

Para criar dois índices temporários, utilize o código abaixo:

```
dbSelectArea("SE1")
cIndex      := CriaTrab(nil,.f.)
cIndex2     := CriaTrab(nil,.f.)
cChave      := IndexKey()
IndRegua("SE1",cIndex,"E1_FATURA+E1_NUM+E1_SERIE",,,OemToAnsi("Selecioneando Registros..."))
IndRegua("SE1",cIndex2,"E1_NUM",,,OemToAnsi("Selecioneando Registros..."))
nIndex := RetIndex("SE1")
dbSelectArea("SE1")
#IFDEF TOP
    dbSetIndex(cIndex+OrdBagExt())
    dbSetIndex(cIndex2+OrdBagExt())
#ENDIF
dbSetOrder(nIndex+1)
dbSetOrder(nIndex+2)

...

dbSetOrder(nIndex+1)
dbSeek(M->mv_par01,.T.)

While SE1->E1_FATURA <= M->mv_par02 .AND. !SE1->(Eof())
...

dbSetOrder(nIndex+2)
dbSeek(M->mv_par02,.T.)

While SE1->E1_NUM <= M->mv_par02 .AND. !SE1->(Eof())
...
```

Criando e Deletando Arquivos temporários

Quando criamos um arquivo de trabalho ou um índice de trabalho (utilizando a função *Indregua*) é obrigatório que sejam apagados ao final do programa.

Exemplo:

Para criação de um índice de Trabalho (Temporário) com *Indregua*:

```
cArqTmp := CriaTrab( NIL, .T. )           //Criando Arquivo
.....

dbCloseArea()
Ferase(cArqTmp+GetdbExtension())         // Deletando o arquivo
Ferase(cArqTmp+OrdBagExt() )             // Deletando índice
```

Importante!!

Utilize a função *GetdbExtension()* para retornar a extensão do arquivo de trabalho. Não utilize “.dbf”, “.dbt” etc como mostrado abaixo:

```
Ferase(cArqTmp+”.dbf”) ←----- Incorreto!
Ferase(cArqTmp+GetdbExtension()) ←----- Correto!
```

Usando Filtros

A utilização de filtros poderá ser feita de duas maneiras:

■ Set Filter to

```
dbSelectArea("CV3")
cFilCV3 := xFilial("CV3")
Set Filter to CV3->CV3_FILIAL == cFilCV3 .AND. CV3->CV3_DTSEQ == dDtCV3 .AND.;
        CV3->CV3_SEQUEN == cSequenc
.....
dbSelectArea("CV3")
Set Filter to
EndIf
```

■ IndRegua()

```
cIndex      := CriaTrab(nil,.f.)
cChave      := IndexKey()
cFiltro := 'E1_FILIAL=="'      +cFilial + '".And.'
cFiltro += 'E1_FATURA=="'      +cFatura + '".And.'
cFiltro += 'E1_TIPOFAT=="'      +cTipo      + '""'

IndRegua("SE1",cIndex,cChave,,cFiltro,OemToAnsi(STR0048))
nIndex := RetIndex("SE1")
dbSelectArea("SE1")
#IFDEF TOP
        dbSetIndex(cIndex+OrdBagExt())
#ENDIF
dbSetOrder(nIndex+1)
dbGoTop()
```

Ao término do uso do filtro o mesmo deverá ser desabilitado, utilizando-se uma das seguintes funções / comandos:

- dbSetFilter()
- Set Filter to
- dbClearFilter()

Querys – Embedded SQL

O objetivo do Embedded SQL é facilitar a escrita e leitura de query's. Foi definida uma sintaxe para que se possa escrever a query diretamente no código ADVPL, sem a necessidade de ficar concatenando pedaços de string para compor a string final.

Recomenda-se que novas querys sejam desenvolvidas utilizando-se este novo recurso.

Exemplo:

Query padrão:

```
cQuery : 'SELECT SE2.E2_PREFIXO,SE2.E2_NUM '  
cQuery += 'FROM '+RetSqlTable('SE2')+' SE2,'+RetSqlTable('QEK')+'  
QEK '  
cQuery += 'WHERE SE2.E2_FILIAL= '+xfilial('SE2')+' AND '  
cQuery += 'SE2.E2_PREFIXO<> ''+cPrefixo+' ' AND '  
cQuery += 'SE2.D_E_L_E_T_ = ' ' '  
cQuery += 'ORDER BY '+RetSqlOrder('SE2')  
  
dbUseArea(.T., 'TOPCONN', TcGenQry(, , cQuery), 'E2TEMP', .T., .T.)  
  
TCSetField('E2TEMP', 'E2_EMISSAO', 'D', 8, 0)
```

Embedded SQL:

```
BeginSql alias 'E2TEMP'  
    column E2_EMISSAO as Date  
    %noparser%  
    SELECT SE2.E2_PREFIXO,SE2.E2_NUM  
    FROM %table:SE2% SE2,%table:QEK% QEK  
    WHERE SE2.E2_FILIAL= %xfilial:SE2% AND  
           SE2.E2_PREFIXO<> %exp:cPrefixo% AND  
           SE2.%notDel%  
    ORDER BY %Order:SE2%  
EndSql
```

Outro Exemplo:

```
BeginSql alias 'E2TEMP'
  column E2_EMISSAO as Date, E2_VALOR as Numeric(tam_cp,2)
  column QEK_SKLDOC As Logical

  %noparser%

SELECT SE2.E2_PREFIXO, SE2.E2_NUM, SE2.E2_FORNECE, SE2.E2_LOJA,
SE2.E2_VALOR, SE2.D_E_L_E_T_ DEL1, QEK.D_E_L_E_T_ DEL2 ,
QEK.QEK_SKLDOC, SE2.R_E_C_N_O_ SE2RECNO
  FROM %table:SE2% SE2,%table:qek% QEK
  WHERE SE2.E2_FILIAL= %xfilial:SE2% AND
        gek.%notDel% and
        SE2.E2_PREFIXO<> %exp:cPrefixo% AND
        SE2.E2_NUM<> %exp:(cAlias)->M0_CODIGO% AND
        SE2.E2_NUM<>45          AND
        SE2.E2_FORNECE=%exp:Space(Len(SE2->E2_FORNECE))% AND
        SE2.E2_EMISSAO<>%exp:MV_PAR06% AND
        SE2.E2_LOJA<>%exp:MV_PAR05% AND
        SE2.E2_VALOR<>%exp:MV_PAR04% AND
        gek.QEK_SKLDOC<>%exp:MV_PAR03% And
        SE2.%notDel%
  ORDER BY %Order:SE2,1%
EndSql
```

Dicas de Tuning

Integridade Referencial

Chaves Primárias

Chaves Estrangeiras

Dicas Importantes

Sistema Internacionalizado

A TOTVS atua hoje em diversos países e isto faz com que os seus sistemas precisem estar adaptados às realidades fiscais e comerciais de cada um desses países. Para que isso seja aplicável, o sistema precisa ser traduzido e localizado.

Diferença entre Localizar e Traduzir

Localizar

De maneira geral, **localização de software** é a adaptação de um sistema já desenvolvido para um determinado país ou região que precisa de adequações legais e de regras de negócios para um outro país ou região. Sob esse ponto de vista, também podemos afirmar que qualquer regra de negócio que atenda exclusivamente ao Brasil, pode ser considerada como uma localização Brasil.

A localização de um sistema independe da língua na qual ele será executado. Dessa forma é possível termos um sistema localizado para a Argentina que será executado em inglês.

Veja abaixo as principais dúvidas sobre o processo de localizar:

- O cálculo de um determinado imposto de um país, assim como todas as necessidades legais que esse imposto estabelece (consultas e relatórios) é uma localização.
- O parâmetro MV_PAISLOC configura a localização que foi instalada. Esse parâmetro carrega a variável pública cPaisLoc, que pode ser usada nos programas, com a sigla do país.
- Uma regra de negócio não presente no ERP padrão Brasil, mas praticada em outro país (de forma genérica por todas as empresas) é uma localização.
- Um programa que valide o "RUT" no Chile (equivalente ao CNPJ/CPF no Brasil) é uma localização.
- Se um cliente na Argentina quiser usar o sistema em português, terá todos seus impostos locais calculados corretamente, uma vez que a localização é independente da língua na qual o sistema está sendo usado.
- Nem todo módulo implantado em outro país deve ser localizado.

Traduzir

A **tradução de software** verte as mensagens exibidas pelo sistema para outra língua. Atualmente o Protheus é comercializado em 3 línguas: Português, Espanhol e Inglês.

O fato do sistema ser executado numa determinada língua, não significa que o mesmo possui alguma localização.

Como produzir um programa traduzido?

Todas as mensagens que permitam interação com o usuário deverão estar traduzidas para as 3 línguas na qual o sistema opera. Isto é possível a partir de duas ferramentas básicas:

Arquivos "header" (CH) anexados ao código-fonte

Veja o exemplo abaixo:

```
#Include "Protheus.ch"
#include "PrgExem.ch"
/*
```

Programa	PrgExem	Microsiga	Data	xx.xx.xx
Descrição	Programa de exemplo -> Entrada de Valor			
Sintaxe	PrgExem()			
Parametros				
Retorno	nValor = Valor Informado			
Uso	Generico			
ATUALIZACOES SOFRIDAS DESDE A CONSTRUÇÃO INICIAL.				
Programador	Data	BOPS	Motivo da Alteracao	

```
*/
Function PrgExem()

Local nValor := 0
Local oDlg

DEFINE MSDIALOG oDlg FROM 31,15 TO 510,574 TITLE OemToAnsi(STR0001) PIXEL OF oMainWnd // "Entrada de Valor"

@ 010, 010 SAY OemToAnsi(STR0002) SIZE 18,07 OF oDlg PIXEL // "Informe Valor"
@ 010, 030 MSGET nValor SIZE 57,10 OF oDlg PIXEL Valid(nValor>0)

DEFINE SBUTTON FROM 06, 246 TYPE 1 ENABLE OF oDlg ACTION { nOpcl := 1,oDlg:End() }
DEFINE SBUTTON FROM 20, 246 TYPE 2 ENABLE OF oDlg ACTION { nOpcl := 0,oDlg:End() }

ACTIVATE MSDIALOG oDlg

Return (nValor)
```

Mensagem exibida
em tela

Arquivo “PrgExem.ch”:

```
#ifndef SPANISH
#define STR0001 "Entrada del Valor"
#define STR0002 "Informe el Valor"
#else
#ifdef ENGLISH
#define STR0001 "Value Input"
#define STR0002 "Input Value"
#else
#define STR0001 "Entrada de Valor"
#define STR0002 "Informe Valor"
#endif
#endif
#endif
```

Para criar um fonte traduzido são necessários:

■ #Include “PrgExem.ch”

É a chamada do arquivo que contém as traduções das mensagens (strings) mostradas ao usuário. As constantes (STR’s) presentes neste arquivo serão anexadas à aplicação durante a compilação do código-fonte.

■ STR0001 / STR0002

Constante que será trocada em tempo de compilação pelo conteúdo presente no arquivo PrgExem.Ch. No exemplo mostrado, ao executar a aplicação em **Inglês**, a constante STR0001 será exibida como “Value Input”.

Importante!!

- Observe que no código-fonte existe o comentário: //”Entrada de Valor”. Este é apenas um indicativo do que será mostrado para o usuário. Comentários deste tipo são obrigatórios para facilitar a leitura do código-fonte.
- Utilize sempre a função OemToAnsi(STRxxxx) para exibir as mensagens.
- A criação dos arquivos CH de traduções é feita a partir da ferramenta ATUSX. Não crie diretamente arquivos CH, pois os mesmos não serão traduzidos! A área de desenvolvimento pode apenas cadastrar STR’s em português, as demais serão traduzidas pela equipe de Traduções através da própria ferramenta ATUSX.

Dicionário de Dados

Todo o dicionário de dados do sistema é traduzido. Observar os seguintes pontos:

- Quando criado um campo novo, ou modificado o conteúdo de um já existente, os campos que devem refletir esta alteração nos demais idiomas devem ser deixados em branco, assim é como o pessoal de traduções identifica os campos que devem ser traduzidos. Isto é válido para todos os arquivos do dicionário de dados.
- Quando criado ou alterado um novo HELP (de campo ou de programa) deve ser informado de imediato para traduções para proceder a tradução para os outros idiomas.

- Para atualizar um parâmetro deve ser sempre usada a função PUTMV, NUNCA DEVE SER PREENCHIDO NEM POSICIONADO POR FORA. Esta função atualiza nos três idiomas.
- Os novos campos tipo COMBO, devem ser criados com numeração e não com siglas (1 para sim e 2 para não, ao invés de S para sim e N para não).
- Quando for criado um novo parâmetro, ou modificado o conteúdo default de um já existente, esta modificação deve ser aplicada nas 3 línguas.
- Quando houve a possibilidade de pegar um STR do dicionário (função RETTITLE()), este deve ser pego, o que evita ter que criar vários STR e tratarmos com a variável cPaisLoc dentro do programa. Exemplo CGC, NOTA FISCAL, CEP, etc.
- Não deve ser usada a acentuação

Como Produzir um programa Localizado

Necessidade de localizar

- Quando criado um campo novo de uso exclusivo de Brasil (E1_INSS por exemplo) deve ser informada a equipe de localizações para configurar este campo (uso, browse, etc.) de acordo com os demais países.
- Quando for modificada a característica de um campo do sistema e este estiver replicado para o resto dos países, as alterações devem ser replicadas em todos os países. Na dúvida da aplicabilidade da alteração nos outros países, deve ser informada a equipe de localizações.

Processamento Automático

Rotinas Automáticas

O que são?

A cada dia estamos criando rotinas com interface automática para melhorar a entrada de dados via outros equipamentos, tais como coletores de dados, interface de outros softwares, etc. Porém, para nossa própria portabilidade e utilização de rotinas padronizadas, temos adotado o próprio programa standard, contudo sem interferência do usuário (digitador). Para tal, criamos um mecanismo onde todos os programas que necessitem desta regra devem ser capazes de “inserir” dados de forma automática. Abaixo mostraremos como proceder :

Como fazer?

Tome como exemplo o MATA250.PRX . O vetor aRotAuto é passado para o programa citado. Se este vetor contiver elementos, significa que será utilizada a Rotina Automática. Este vetor deve, quando da utilização das rotinas automáticas, conter os dados mínimos necessários para a atualização dos arquivos.

Veja a estrutura do vetor a ser enviado para a rotina automática.

ARotAuto := { cCampo, Conteúdo, Validação }

Onde

CCampo -> é o campo a ser atualizado,

Conteúdo -> é o conteúdo que cCampo vai receber

Validação -> é a validação que cCampo vai receber.

Observação: A Validação pode ser uma função ou um valor 'NIL'. Se for 'NIL', as validações a serem utilizadas para o respectivo campo serão as existentes no SX3. Se as validações não forem as do SX3, elas devem ser passadas numa função.

Exemplo:

```
ARotAuto := { { "D3_TM" ,"001" ,NIL } , ;  
              { "D3_COD" ,pdr("10100",15) ,NIL } , ;  
              { "D3_UM" ,"UN" ,NIL } , ;  
              { "D3_QUANT" ,1 ,NIL } , ;  
              { "D3_OP" ,"00000401001" ,NIL } , ;  
              { "D3_LOCAL" ,"01" ,NIL } , ;  
              { "D3_EMISSAO" ,dDataBase ,NIL } }
```

Para o processo de inclusão simples, sem getdados, a variável padrão a ser utilizada nos programas chama-se aRotAuto, e para processo de inclusão com cabeçalho e itens, as variáveis a serem utilizadas são: aAutoCab para o cabeçalho, e aAutoItens para os itens da getdados.

Para uma inclusão simples, tomar como exemplo o MATA250.PRX. Para uma inclusão com cabeçalho e item, tomar como exemplo o CONA050.PRX.

Schedule

Recomendação: Processos longos, que não dependem de interferência de usuário, recomendamos que sejam executados como JOB (Reprocessamentos, acertos, etc). Motivo: Se durante o processamento a conexão entre Protheus Remote e Protheus Server cair, ocorrerá perda do resultado do processamento, ou gerar inconsistências na base de dados.

Processos de Integração

Em construção

Customizações

Pontos de Entrada

O que são?

Os pontos de entrada tem por objetivo deixar o sistema flexível, permitindo uma grande variedade de desenvolvimento pelos nossos analistas de suporte de acordo com a necessidade de cada tipo de cliente/implantação.

Quando criar?

- Avaliar com critério a criação do Ponto de Entrada, pois é importante inseri-lo num ponto que seja útil, não redundante e que realmente dê condições de atender ao solicitante.
- O Ponto de entrada não pode ser usado como uma ferramenta de correção de eventuais falhas do sistema.
- Em processos críticos do sistema NÃO devem ser criados pontos de entrada, pois poderá tornar os resultados do sistema totalmente imprevisíveis.
- Todo novo ponto de entrada deve ser documento no DEM.

Utilização

Duas funções são primordiais e obrigatórias para o funcionamento de um ponto de entrada:

- **ExistBlock():** Verifica a existência ou não do ponto de entrada no repositório. Sua utilização é obrigatória, e deve condicionar a execução do ponto de entrada. O seu retorno poderá ser .T. no caso do ponto de entrada existir ou .F. caso contrário.
- **ExecBlock():** Executa o ponto de entrada. O ideal é efetuar o teste da existência do mesmo pela função Execblock(), antes de tentar a sua execução.

```
Function TMKA010()  
  
Local lRet := .F.  
Local lPE := Existblock("TMKMCI")  
If lPE  
    lRet := Execblock("TMKMCI", .F., F., {aValor})  
Endif  
Return(lRet)
```

Importante!

Efetuar o teste da existência apenas uma vez no código-fonte, para não sobrecarregar o processamento.

Semáforo

O controle de Semaforo permite que o sistema controle a Numeração Automática de Documentos On Line. Temos basicamente 3 funções que gerenciam o controle do mesmo. São elas :

- **GETSXENUM(EXPC1)** -> Obtem o número sequencial do alias especificado no parâmetro.
- **ROLLBACKSXE** -> Descarta o número pendente do semáforo. É usado quando o usuário cancela a operação (o numero não é aproveitado).
- **CONFIRMSXE** -> Confirma o número sugerido. Esta função deve ser chamada quando da confirmação da gravação do registro.
- **MAYIUSE** -> Checa fisicamente se um determinado arquivo existe. O arquivo poderá conter o número sequencial.

Obs : A função GETX8NUM executa a própria GETSXENUM.

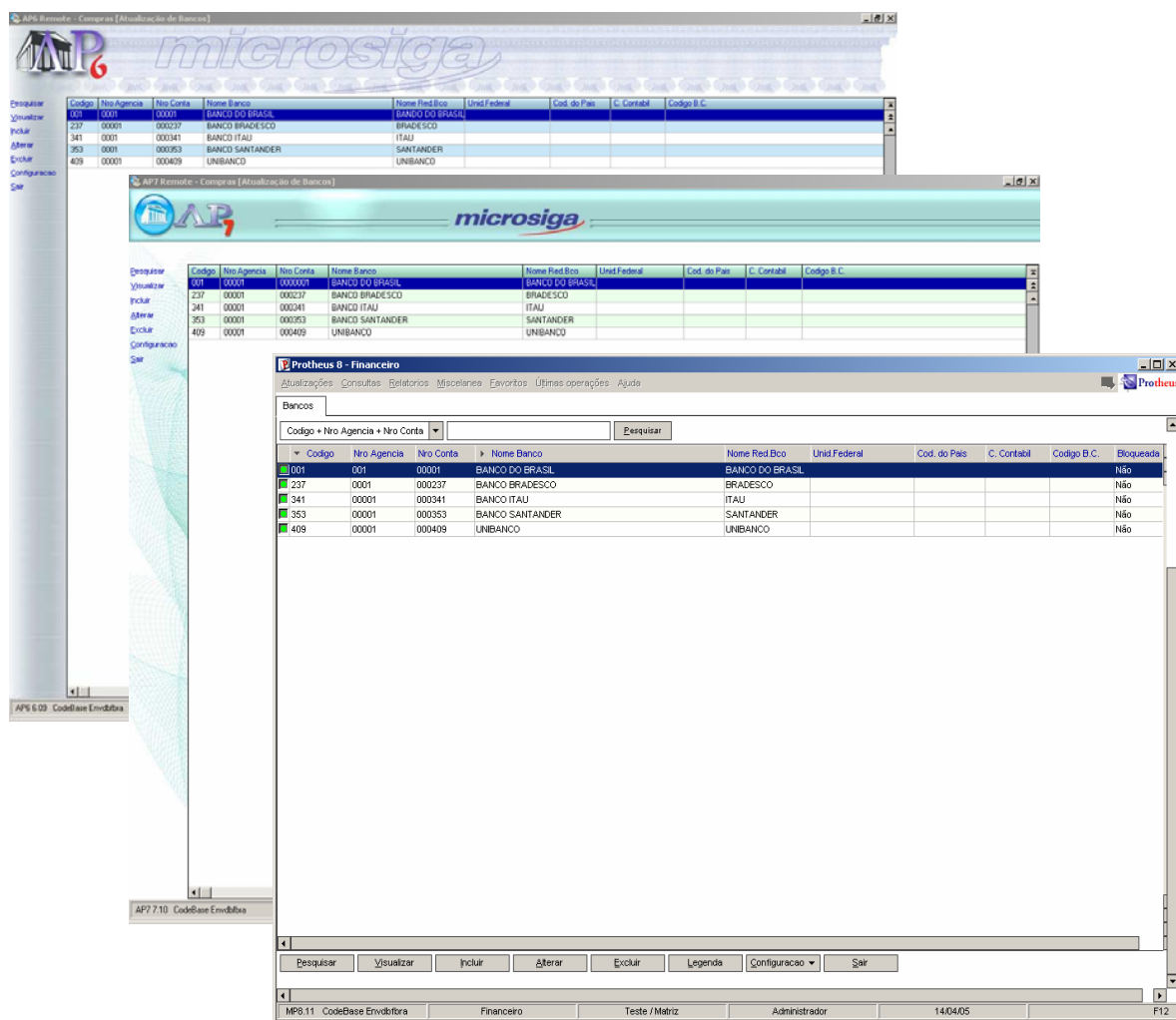
Desenvolvendo Telas

A padronização de telas é de fundamental importância na linha de aprendizagem de utilização do sistema, pois se você aprender a utilizar um cadastro, saberá utilizar todos os outros.

Interfaces do Protheus

No decorrer das versões e releases do produto Microsiga, as interfaces sofreram modificações para receber melhorias e visuais diferenciados. Exemplo da evolução:

Abaixo temos, respectivamente, a mesma tela de cadastro de Bancos nas Interface Classic (AP6), Ocean (AP7) e Flat (MP8).



Independente da evolução de interface, o produto mantém seus facilitadores de padronização, ou seja, componentes de código que se adaptam à tecnologia e à própria mudança de interface. Alguns exemplos deles são: mBrowse(), Enchoice(), MsGetDados() etc.

Browsets

mBrowse()

Protheus 8 - Compras

Atualizações Consultas Relatórios Miscelanea Favoritos Últimas operações Ajuda

Bancos

Codigo + Nro Agencia + Nro Conta Pesquisar

Codigo	Nro Agencia	Nro Conta	Nome Banco	Nome Red.Bco	Unid Federal	Cod. do Pais	C. Contabil	Codigo B.C.	Bloq
001	00001	0000001	BANCO DO BRASIL	BANCO DO BRASIL					Não
237	0001	000212	BANCO BRADESCO	BRADESCO					Não
341	0001	002121	BANCO ITAU	ITAU					Não
353	0001	000353	BANCO SANTANDER	SANTANDER					Não
409	0001	000409	UNIBANCO	UNIBANCO					Não

SEM FILTRO

Pesquisar Visualizar Incluir Alterar Excluir Legenda Configuracao Filtro Sair

MP8.11 SQL Envtopbra Compras Teste / Matriz Administrador 14/04/05

A mBrowse() é um dos modelos mais comuns e é, praticamente, utilizado por todos os cadastros no sistema. Ela oferece alguns recursos como:

- o Cores para classificação dos registros e montagem da legenda
- o Pesquisa por qualquer índice da tabela em uso
- o Organização por qualquer campo chave (indicado pela seta ao lado do campo nas colunas), bastando clicar na coluna para mudar a ordem
- o Filtro de usuário (mostrado no rodapé do browse)
- o Configuração (seleção de colunas para exibição, restauração do browse original e opção para não exibição do browse)

Exemplo: Cadastro de bancos.

Vide Anexo X – Exemplos de programas

MarkBrow()

Numero da SC	Item da SC	Produto	Unid Medida	Quantidade	Segunda UM	Qtd. 2a UM	Necessidade	Armazen	Observacao
000001	0001	PRD001	UN	120,00		0,00	18/04/05	01	
000001	0002	PRD002	UN	220,00		0,00	18/04/05	01	
000001	0003	PRD003	UN	180,00		0,00	18/04/05	01	
000001	0004	PRD004	UN	160,00		0,00	18/04/05	01	

A MarkBrow() é utilizada para browses onde é necessário fazer uma seleção de elementos para posterior processamento. Geralmente é precedida por um grupo de perguntas onde serão filtrados os dados a serem exibidos.

Exemplo: Geração das cotações de compra.

Vide Anexo X – Exemplos de programas

TWBrowse()

Protheus 8 - Financeiro

Atualizações Consultas Relatórios Miscelânea Favoritos Últimas operações Ajuda

Reconc Bancaria

Reconciliação Bancária - Reconciliacao

Recortar Copiar Colar Calo Agenda Spool Ajuda Editar OK Cancelar

Saldo Anterior	0,00	Saldo Atual	-900,00
Total Recebido	0,00	Total Pago	900,00
Docs.Reconc. Receber	0	Valor Recebido	0,00
Docs.Reconc. Pagar	1	Valor Pago	800,00

Rec	Dt Disponibilidade	Número	Vlr. Movimen.	Natureza	Banco	Agência	Cont.
14/04/05	M1	100,00	001	001	001	0000	
14/04/05	M1	800,00	001	001	001	0000	

MP8.11 CodeBase Envldbfbra Financeiro Teste / Matriz Administrador 14/04/05 F12

A TWBrowse() permite construir um objeto browse similar ao MarkBrow() apresentado anteriormente, porém com a possibilidade de juntá-lo a uma tela com outros componentes de exibição e entrada de dados.

Exemplo: Reconciliação Bancária.

Vide Anexo X – Exemplos de programas

Perguntas

Pergunte()

Parametros

Filtrar por Data ? Sim

Dt emissão Inicial ? 18/04/05

Dt Emissão Final ? 18/04/05

Núm.dias p/validade ? 10

Filtrar SC ja Gerada ? Sim

Utiliza amarração ?

Informe se a geração das cotações devem ser filtradas por data de emissão da solicitação de compra.

Ok Cancelar

A Pergunte() é o facilitador para criação de perguntas. Seja para obter um filtro de browse ou para determinar parâmetros de um relatório ou processamento, ela se baseia no grupo informado e previamente cadastro na tabela SX1. As propriedades desse componente são:

- o Gerar gets conforme o tipo de campo: data, numero, caracter e combo box (lista de opções).
- o Armazenar a ultima resposta informada (SX1). A partir da versão 8.11 essa informação é por usuário (profile).
- o Permitir exibir help por pergunta (rodapé)

Vide Anexo X – Exemplos de programas

Entrada de Dados

Enchoice()

A Enchoice() é o modelo mais utilizado para realizar a manutenção de cadastros do sistema. Seja na inclusão, alteração, consulta ou exclusão, este componente pode ser usado para manipular os campos de uma tabela de dados definida no dicionário (SX3). Algumas das propriedades são:

- o Disposição e alinhamento automático dos campos em duas colunas seguindo a ordem definida no dicionário de dados SX3.
- o Destaque de campos de preenchimento obrigatório e bloqueio para campos somente de leitura.
- o Agrupamento de campos por painéis, definidos no arquivo SXA e associados em cada campo do dicionário.
- o Considera os tipos de campos e suas propriedades (calculadora para números, calendários para datas, listas de opções, F3-pesquisa em outras tabelas etc)
- o Aplicação das propriedades dos campos de dicionário (x3_relacao, x3_usado, x3_valid, x3_cbox, x3_f3 etc) e permite o uso de gatilhos (SX7)

Exemplo: Manutenção do cadastro de Produtos

Para cadastros simples, pode ser utilizado o facilitador `AxCadastro()`, o qual já monta um browse (`mBrowse` citado anteriormente) considerando as opções de inclusão (`AxInclui`), alteração (`AxAltera`), consulta (`AxVisual`) e exclusão (`AxDeleta`). O componente `Enchoice()` é utilizado por todas essas opções para a montagem da tela.

Vide Anexo X – Exemplos de programas

MSGetDados()

Item	Produto	Unidade	Segunda UM	Quantidade	Tab. Preço	Preço Unitário	Vlr. Total	Gtd. 2a UM	Aliq. IPI	Numero da SC	Item da SC	Dt. Entrega
0001	PRD001	UN		120,00		15,00	1.800,00	0,00	0,00 000001	0001	18/04/05	
0002	PRD002	UN		220,00		18,00	3.960,00	0,00	0,00 000001	0002	18/04/05	
0003	PRD003	UN		180,00		22,00	3.960,00	0,00	0,00 000001	0003	18/04/05	
0004	PRD004	UN		160,00		26,00	4.160,00	0,00	0,00 000001	0004	18/04/05	

Totais	Inf. Fornecedor	Frete/Despesas	Descontos	Impostos	Mensagem/Reajuste
Valor da Mercadoria	13.880,00		Descontos	0,00	

A MSGetDados() é o componente mais utilizado para a construção de grids de digitação/exibição. Algumas das propriedades são:

- o Inclusão, alteração e exclusão de linhas
- o Inclusão de número sequencial automático (identificador de item)
- o Considera os tipos de campos e suas propriedades (calculadora para números, calendários para datas, listas de opções, F3-pesquisa em outras tabelas etc)
- o Validação por linha e validação após todo o preenchimento
- o Aplicação das propriedades dos campos de dicionário (x3_valid, x3_cbox, x3_f3 etc) e permite o uso de gatilhos (SX7)

Exemplo: Itens do Pedido de Compras

Ao compor uma tela, além da área de linhas ou itens -MSGetDados()-, é comum definir-se um cabeçalho ou rodapé. Para esses casos existem dois facilitadores: a Modelo2() e a Modelo3(). Ambas estão definidas no programa matxatu.prx e podem ser usadas como referência.

A Modelo2() é capaz de criar uma tela com cabeçalho fixo (campos passados através de um array), um grid de itens – MSGetDados() - e um rodapé (campos também passados através de um array)

A Modelo3() difere da Modelo2() na construção do cabeçalho, pois permite um cabeçalho obedecendo as regras de uma Enchoice() referente à alguma tabela do dicionário de dados.

Vide Anexo X – Exemplos de programas

Componentes Individuais

TSay()

Componente que permite a exibição de algum texto na tela, podendo ser definida a posição, tamanho, picture, cor, estilo de fonte a ser usado, entre outros.

Exemplo: Consulta Histórico do Produto



TGet()

Permite a construção de um elemento de entrada de dados, podendo-se definir: posição de tela específica, estilo de fonte, cor, picture de entrada, validação, entre outros.

Intervalo de ?

Intervalo ate ?

Recurso ?

Exemplo: Tela de Filtro para o gerenciamento de Apontamentos no PMS

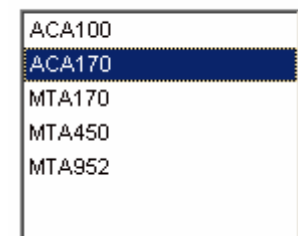
TComboBox() e TListBox()



O TComboBox() e o TListBox() permitem a construção de um elemento de entrada de dados com uma lista de opções pré-definida. A diferença entre eles é que o

TComboBox() ocupa menos espaço na tela, sendo que a sempre há uma opção da lista pré-selecionada, por default. Já no TListBox() pode-se definir o espaço de forma a visualizar parte ou todas as opções disponíveis.

Pergunta(s) Customizada(s)



Exemplos: Compensação entre Carteiras e Cadastro de perguntas customizadas por usuário

TCheckBox()

Permite a construção de um elemento de entrada de dados com uma lista de opções, onde pode ser selecionado uma, mais de uma ou nenhuma opção através de uma marca.

Exemplo: Opções do Fluxo de Caixa

- ☒ Pedido de Venda
- ☒ Pedido de Compra
- ☐ Aplicacoes/Emprestimos
- ☐ Saldos Bancarios
- ☐ Titulos em Atraso

TButton() e SButton()



Permite a construção de botões na tela. Pode ser definida: posição, tamanho, texto e estilo de fonte para o TButton() ou tipo pré-definido para o SButton(), ação, entre outros.

Exemplos: Botões da Consulta Histórico de Produtos e Botões da Manutenção de Estrutura de produtos.

TPanel()

Código	Loja	Nome
0001	01	CLIENTE TESTE DE BASE

Produtos/Eqto	Descrição
PRD002	PRODUTO DOIS TEST
Dt.Garantia	Contrato de Manutenção
02/08/05	

Componente que permite a construção de um painel para separar, agrupar, organizar grupos de textos, browses, entrada de dados, perguntas etc.

Vide Anexo X – Exemplos de programas

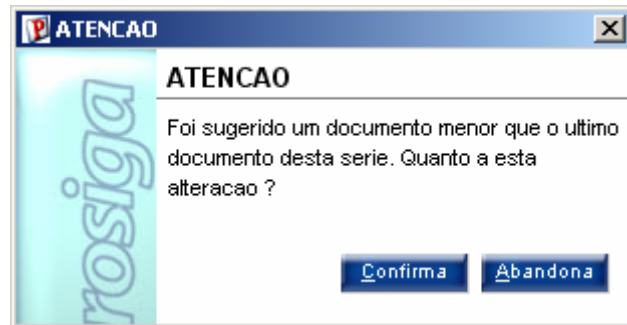
Mensagens

Aviso()

A função Aviso() gera uma mensagem onde podem ser definidos:

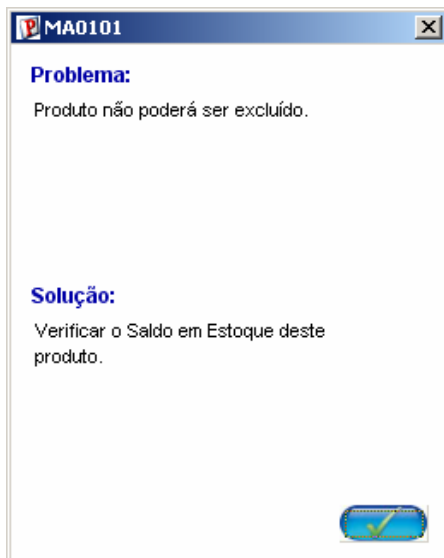
- o O título
- o O corpo da mensagem
- o Os botões de resposta

Os botões a serem exibidos são definidos em um array. A função retornará o índice do array conforme a escolha feita. No exemplo, retornará 1 caso seja escolhida a opção 'Confirma' e 2 caso seja escolhida a opção 'Abandona'.



Help()

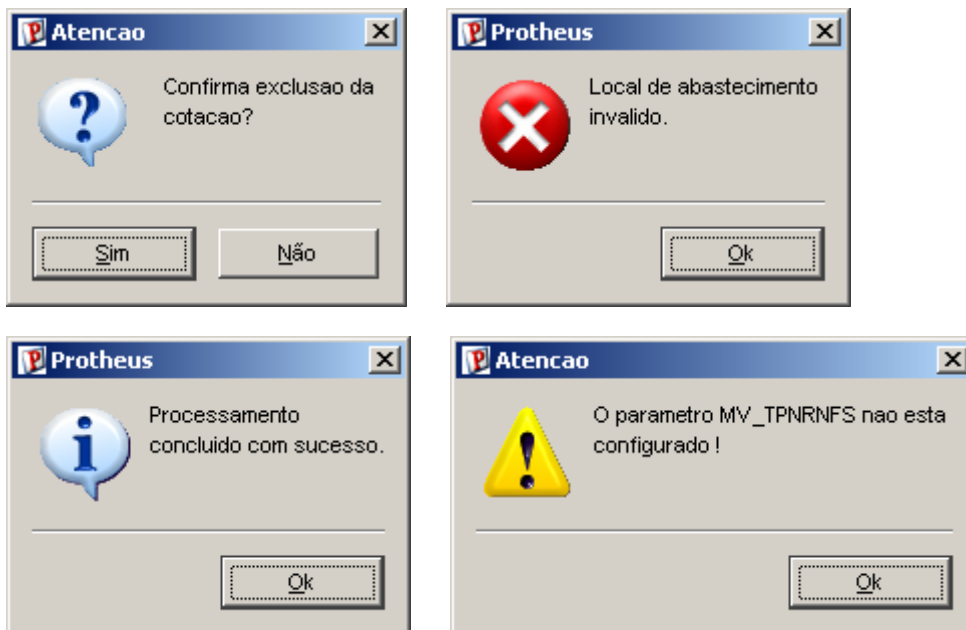
A função Help() é utilizada, geralmente, para mensagens de alertas ou erros, onde pode ser definida uma solução para um determinado problema. Ela também é usada como default dos helps de campo presentes no sistema. As descrições dos problemas e soluções para todos os helps são carregados na instalação ou atualização de versão. Também é possível, via programa, criar novos helps utilizando a função PutHelp().



A função Help() permite que se defina:

- Qual help será utilizado (para trazer a descrição do problema e solução para a tela)
- Complemento da mensagem
- Linha e coluna onde será exibido o complemento da mensagem

MsgNoYes(), MsgStop(), MsgInfo() e MsgAlert()

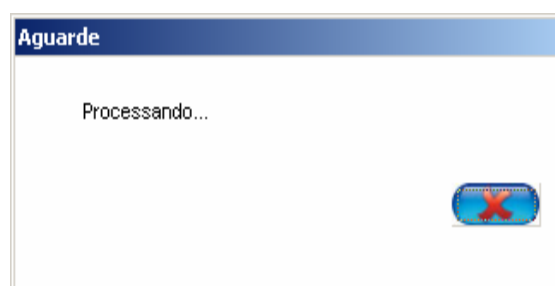


Este grupo de mensagens exibem, através de símbolos, o alerta respectivo para cada tipo:

- MsgNoYes() - Tipo escolha/pergunta (opções Sim ou Não – retorno booleano)
- MsgStop() - Tipo parada/erro (opção Ok)
- MsgInfo() - Tipo informação (opção Ok)
- MsgAlert() - Tipo atenção/alerta (opção Ok)

MSAguarde()

A função `MSAguarde()` é utilizada, geralmente, na execução de processos forma que seja mostrada uma mensagem de início do processo. Após exibição inicial pode-se controlar o progresso do processamento. Tal progresso pode ser exibido na tela através do uso de outras funções como `IndRegua()`, `SetRegua()`, `IncProc()` e `ProcRegua()`.



de
essa

MsNewProcess()

A função `MsNewProcess()` tem a mesma finalidade da `MSAguarde()`, com a possibilidade de controle de um progresso principal - `IncRegua1()` - e um sub-progresso - `IncRegua2()` -.

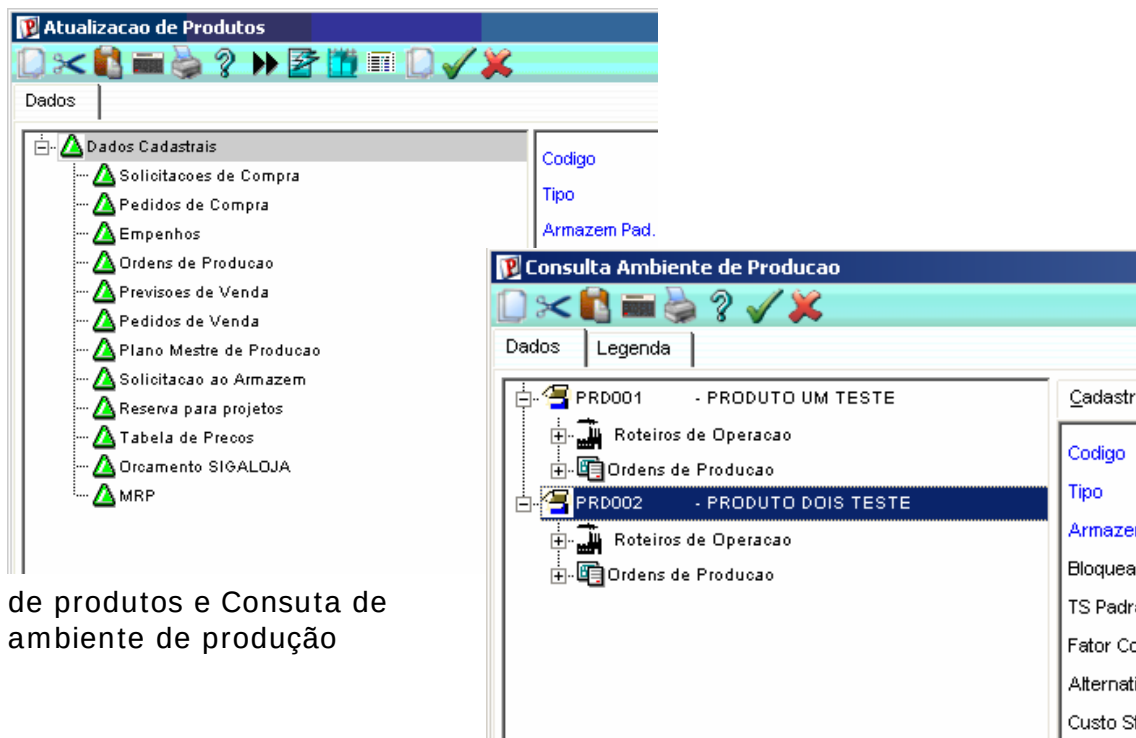
Vide Anexo X – Exemplos de programas

Outros Modelos

dbTree()

A função dbTree() é utilizada na visualização de elementos em forma de árvore. É um facilitador para a exibição de estruturas em níveis, permitindo o uso de imagens para identificação dos itens e definindo ações no momento do posicionamento em algum item.

Esta função é utilizada geralmente para conceitos de estrutura de produtos, contas contábeis, rastreabilidade de processos etc...



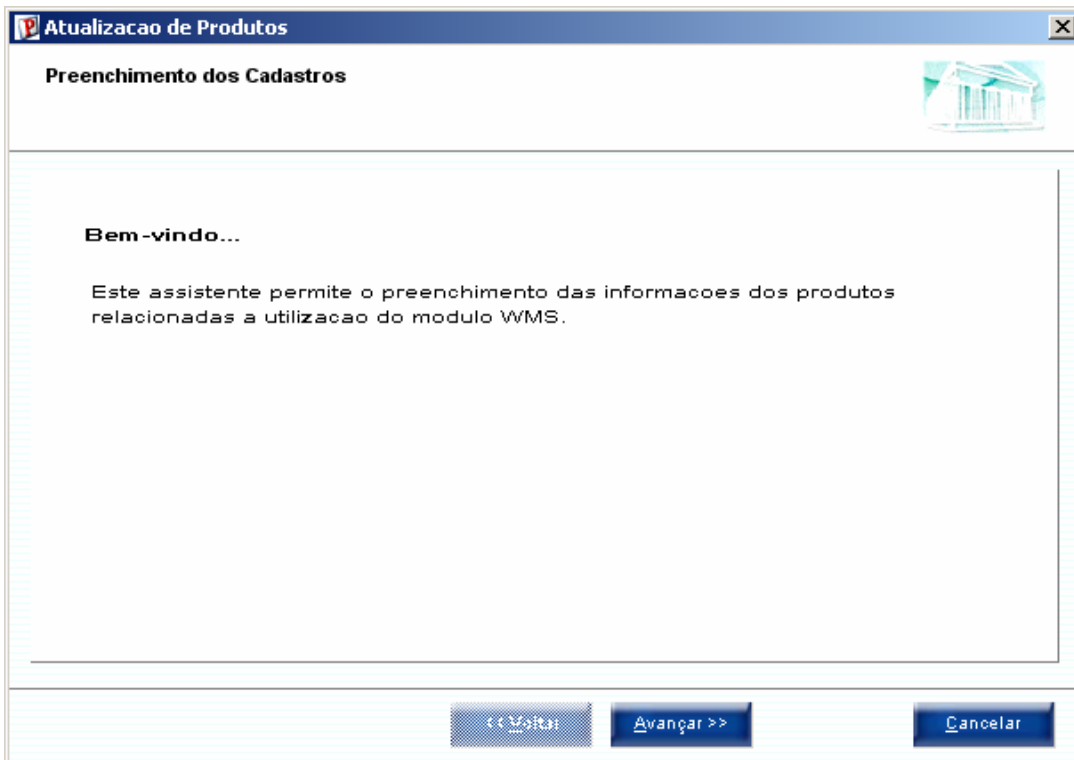
Alguns
exemplo
s:
Consulta

de produtos e Consuta de
ambiente de produção

Vide Anexo X – Exemplos de programas

APWizard()

A APWizard() é usada para a elaboração de telas passo-a-passo no sentido de garantir que determinados procedimentos não sejam esquecidos ou ignorados. Nas telas passo-a-passo podem ser utilizados quaisquer objetos e habilitar o avanço ou retrocesso dependendo do preenchimento.



Exemplo: Integração WMS – atualização dos dados de produtos

Vide Anexo X – Exemplos de programas

Ícones e Legendas

Legendas

As legendas auxiliam na compreensão dos símbolos, cores ou objetos mostrados na tela sem sua descrição explícita. Alguns exemplos de legendas no sistema:

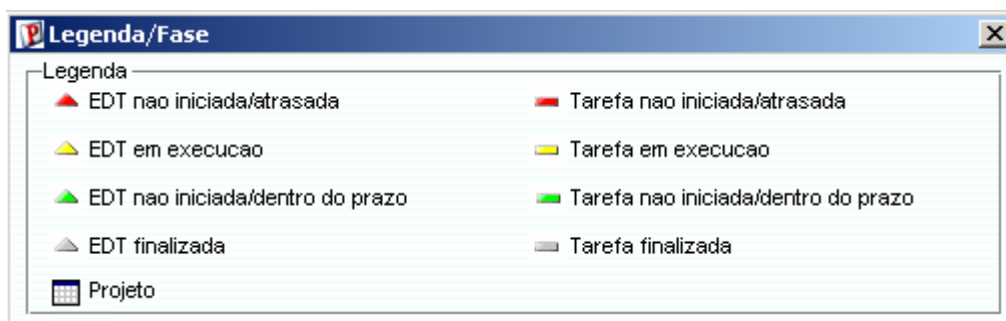
-  Produtos
-  Componentes / Empenhos
-  Consumos
-  Roteiros de Operacao
-  Operacoes
-  Recursos
-  Ferramentas
-  Ordens de Producao

Exemplo: Legenda na Consulta de ambiente de produção

Exemplo: Legenda da Carteira contas a pagar



de



Exemplo: Tela de Legenda/Fase na gerencia de apontamento no PMS

Ícones

ÍCONES DA APLICAÇÃO

O Protheus possui ícones que sempre são os mesmos não importa em que tela se está...
Vide lista de ícones no anexo Y

Desenvolvendo Relatórios

Relatórios Gráficos

Em construção

Pecados da Programação

Excesso de Refresh

Excesso de Refresh nos componentes, principalmente em processamento. Isto inclui mensagens que ficam metralhando o canal de comunicação, principalmente em processament.

Exemplo:

Atualização das mensagens de STATUS com muita frequência, para informar sobre o atual status de processamento.

Nada impede que isto seja feito, porém cabe ao programador usar de bom senso para não fazer notificações desnecessárias. Para casos de necessidade desta informação na interface, uma boa saída é criar uma variável local, numérica, e ir incrementando a variável, e apenas fazer a notificação da interface quando uma porção significativa de dados, que justifique a atualização da interface, tenha sido processada.

SET CENTURY / DATE FORMAT e Loops

Os comandos SET CENTURY ON e SET DATE FORMAT modificam a maneira de exibição de campos data no código ADVPL e também no Protheus Remote. Evitar chamar os comandos em loops. Motivos: Geram tráfego de rede excessivo entre Protheus Server e Protheus Remote. O mesmo vale para as chamadas __SetCentury(<x>), Set(4 /*_SET_DATEFORMAT*/, <c>), Set(5 /*_SET_EPOCH */ , <year>)

Begin...End Sequence / Break

Embora não influencie em performance, programar com Begin Sequence / End Sequence / Break é horrível, e pode trazer efeitos colaterais sérios ! As instruções Begin / End Sequence foram criadas para proteger determinados blocos de código Advpl, para funções especiais que requerem um tratamento de erro específico. Deve ser usado apenas em casos especiais, e SEMPRE que utilizado, DEVE ser escrito com um tratamento de erro apropriado, que salve e restaure o codeBlock de tratamento padrão em uso!

Programar controlando o desvio de execução do programa por BREAK não se faz. Os tratamentos de erro do Framework Advpl foram montados partindo da premissa que um BREAK executado pelo tratamento de erro da LIB, vai cair num RECOVER ou END SEQUENCE da LIB, e não cai, o que pode acabar “engolindo” erros de programação, dependendo do ambiente envolvido.

Exemplo:

Fazer um begin sequence / end sequence, dentro de um bloco transacionado (begin / end transaction) : Caso ocorra um erro dentro do begin sequence, o BREAK executado vai cair no End Sequence, e não no End Transaction!

Interface durante transação

Não se deve colocar parada de Interface entre o Begin Transaction e o End Transaction, como por exemplo, perguntar alguma informação ou decisão ao usuário. As pré-validações e entradas de dados para um determinado processamento em transação devem ser realizadas antes da transação, para que o bloco transacionado não "pense" ou "valide", apenas "faça", para que seja mínimo o tempo que um ou mais registros sejam "retidos" dentro de uma transação.

Transações muito longas

Uma transação longa no código-fonte, é problema na certa. Pense sempre na possibilidade de transações mais curtas, a exemplo: Pedido de Venda que era por bloco e passou a ser por linha. Reforça o princípio que zela para que seja mínimo o tempo que um ou mais registros sejam "retidos" dentro de uma transação.

Alto acoplamento com Interface.

Programar entre a criação do objeto do dialogo e o activate dificulta a transformação da funcionalidade em Web Service. Num conceito mais amplo, vamos enfatizar a separação entre processamento e interface.

dbGoTop

Fazer DbGoTop() após abrir uma query é totalmente desnecessário ! Ao ser aberto um *alias* de uma *query*, ele já está posicionado no primeiro registro. Fazer um DbGoTop() em um *alias* de Query força o re-envio da query ao banco, dobrando o processamento desnecessariamente.

Fontes com "economia" de IF

Se uma função retorna um status de sucesso ou falha, isto deve ser tratado. Existem funções que não abortam a aplicação em caso de insucesso, e retornam um status justamente para o programador fazer o tratamento adequado. Não verificar este retorno é um pecado que, geralmente, apresenta problema em ambientes grandes, sob concorrência ou condições adversas. Por exemplo, não verificar o handle de um arquivo após usar fopen() / fcreate(), não verificar se um DbSeek() realmente posicionou no registro procurado, etc ...

“ * ” em query´s

Evitar o uso de * em queries. Usar somente em casos extremos e de real necessidade, pois eles oneram a *performance* da instrução, trazendo campos que nem sempre são necessários, gerando um tráfego de rede maior. E, existe um limite de número máximo de campos retornados em uma query (aqui – verificar). Se você precisa de mais campos do que isso, é mais eficiente a sua *query* retornar apenas os R_E_C_N_O_ (s) dos registros selecionados, e os dados serem lidos diretamente do alias correspondente, sendo realizado o posicionamento através da instrução DbGoto()

Objetos visuais

Objetos visuais em loops

O uso de propriedades ou métodos visuais em loops gera tráfego de dados desnecessário entre Protheus Server e Protheus Remote. Exemplo:

```
While SD2->( !Eof( ) )
  nVal:= nVal+SD2->D2_VAL
  oSay:VarPut( nVal ) << EVITAR
  SD2->( dbSkip() )
End
```

...

```
While SD2->( !Eof( ) )
  nVal:= nVal+SD2->D2_VAL
  SD2->( dbSkip() )
End
oSay:VarPut(nVal) << MELHOR
```

Objetos visuais e o End()

Não utilizar propriedades ou métodos de objetos visuais de uma janela após a chamada do método End() da janela, pois após a chamada do método End() de uma janela, todos os componentes visuais internos contidos na janelas ficam inválidos (são destruídos). Exemplo:

```
// Não utilizar
@ 10,10 BUTTON oBtn "Fechar" ACTION ( oDlg:End(), nVal:= Len( oLbx:altems ) )

// MELHOR
@ 10,10 BUTTON oBtn "Fechar" ACTION ( nVal:= Len( oLbx:altems ), oDlg:End() )
```

Não chamar o método End() de uma janela no comando ON VALID da mesma janela, pois o programa entrará em Loop e causar erro STACK OVERFLOW.

Objetos visuais em Jobs

Nunca utilizar codificar componente visuais em rotinas que serão executadas em JOBS. Motivo: Irá gerar um erro de execução do programa. Soluções:

Retirar toda construção de janela ou chamada a funções que criam janelas ou chamam Alert(), MsgBox() das rotinas que serão executadas em JOBS.

Testar a execução de rotinas em JOBS com a função GetRemoteType(), se a função retornar -1 a rotina está dentro de um JOB.

MSAdvSize() e dimensões de janela

Evitar o uso da função MSAdvSize() para obter as dimensões de uma janela. Motivo: A função não funciona bem em ambiente MDI e ActiveX. Solução: Organizar a disposição dos controles visuais com o uso de painéis (TPanel) e propriedade de alinhamento de controles nAlign.

Codeblocks em componentes visuais

Não incluir codeblocks extensos nos componentes visuais. Motivo: Codeblocks extensos são muito difíceis de debugar. Solução: Criar uma função auxiliar que contenha o conteúdo do codeblock.

DEFINE DIALOG e ACTIVATE

A definição de uma janela (DEFINE MSDIALOG...) **DEVE** ficar dentro da mesma função que a chamada de ACTIVATE. Motivo: Não pode haver DEFINES de janela em sequência, isso gera erro de execução que é extremamente difícil de debugar em ADVPL.

Funções em ON INIT

Não incluir chamadas de função para rotinas de longa duração no método **ON INIT** das janelas. Motivo: Pode dar uma falsa impressão de que a janela travou. Solução: Mova a rotina para antes do início da construção da janela e use a função ProcRegua() e IncProc() para exibir uma janela com uma régua de execução.

DBTree

Nos programas com DBTREE, **NÃO MANIPULEM** o arquivo de trabalho do componente. Motivo: O DBTREE pode apresentar comportamentos não definidos e que são extremamente difíceis de debugar.